МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №5

з дисципліни «Професійний Пайтон»

на тему: «Обробка помилок, файли та серіалізація»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних: exceptions і власна ієрархія винятків, exception chaining, context managers, logging, pathlib, CSV, JSON, YAML, streaming I/O та атомарний запис файлів.

2. Індивідуальне завдання

Варіант 15. Імпорт та експорт результатів наукових експериментів. Поля: `experiment_id`, `name`, `researcher`, `parameter`, `result`, `date`. Перевіряти ID, назву/дослідника, parameter, numeric result і date; allowed parameter list задавати через YAML. Реалізувати streaming import, export valid measurements/results у JSON, strict/tolerant mode, error logging, statistics та atomic output.

3. Постановка задачі

До проєкту experiment-registry потрібно додати модуль імпорту/експорту, який працює з `Path`, потоково читає CSV, підтримує JSON, використовує YAML-конфігурацію, перевіряє записи, має власну ієрархію винятків з exception chaining, використовує `with` і власний context manager, веде журнал, підтримує strict і tolerant режими, рахує статистику, атомарно записує результат, обробляє помилки файлового вводу-виводу та контрольовано завершується при критичних помилках.

4. Хід виконання

4.1. Input data schema та приклад CSV

Таблиця 5.1. Схема вхідних даних

Поле	Тип	Правило перевірки
`experiment_id`	int	Обов’язкове, ціле, > 0
`name`	str	Обов’язкове, непорожнє
`researcher`	str	Обов’язкове, зайві пробіли нормалізуються
`parameter`	str	Один із `validation.allowed_parameters` (YAML), одиниця береться звідти
`result`	float	Число, скінченне, у межах [min_result, max_result]
`date`	date	Формат YYYY-MM-DD, не в майбутньому

Лістинг 5.1. Вхідний файл data/experiments.csv (містить навмисно некоректні записи)

```
experiment_id,name,researcher,parameter,result,date
1,Thermal conductivity of Al alloy,Olena Kovalenko,temperature,293.4,2026-02-03
1,Thermal conductivity of Al alloy,Olena Kovalenko,temperature,297.1,2026-02-04
1,Thermal conductivity of Al alloy,Olena Kovalenko,temperature,301.8,2026-02-10
2,Gas compression test,Taras Hnatiuk,pressure,101.3,2026-02-05
2,Gas compression test,Taras Hnatiuk,pressure,99.8,2026-02-06
3,Battery discharge cycle,Iryna Melnyk,voltage,3.71,2026-02-12
3,Battery discharge cycle,Iryna Melnyk,voltage,3.65,2026-02-13
4,Cryogenic cooling run,Andrii Bondar,temperature,77.2,2026-02-18
5,Signal generator sweep,Maria Shevchuk,frequency,1500,2026-02-20
```

```
6,Invalid result sample,Taras Hnatiuk,pressure,wrong,2026-02-21
7,Heat exchanger efficiency,Olena Kovalenko,temperature,348.9,2026-03-02
7,Heat exchanger efficiency,Olena Kovalenko,temperature,352.2,2026-03-03
8,Missing researcher,,voltage,0.71,2026-03-04
9,Unknown parameter,Andrii Bondar,colour,5.0,2026-03-05
10,Future date sample,Maria Shevchuk,pressure,26.5,2031-01-01
x1,Bad identifier,Maria Shevchuk,pressure,26.5,2026-03-06
11,Electrolyte titration,Iryna Melnyk,concentration,0.25,2026-03-09
12,Negative result,Andrii Bondar,temperature,-5,2026-03-10
3,Battery discharge cycle,Iryna Melnyk,pressure,101.0,2026-03-11
13,Weather balloon ascent,Maria Shevchuk,pressure,26.5,2026-03-20
```

4.2. Структура проєкту

Лістинг 5.2. Структура проєкту lab05 (без пакетів попередніх робіт)

```
lab05/
├── benchmarks/
│   ├── benchmark_search.py
│   ├── benchmark_streaming.py
│   └── import_experiments.py
├── data/
│   ├── experiments.csv
│   └── experiments_sample.csv
├── src/
│   └── experiment_registry/
│       ├── importer/
│       │   ├── __init__.py
│       │   ├── config.py
│       │   ├── exporters.py
│       │   ├── logging_config.py
│       │   ├── readers.py
│       │   ├── services.py
│       │   └── validators.py
│       ├── __init__.py
│       ├── analytics.py
│       ├── config.py
│       ├── data.py
│       ├── decorators.py
│       ├── exceptions.py
│       ├── main.py
│       ├── models.py
│       ├── processors.py
│       └── services.py
├── .gitignore
├── config.yaml
├── pyproject.toml
└── README.md
```

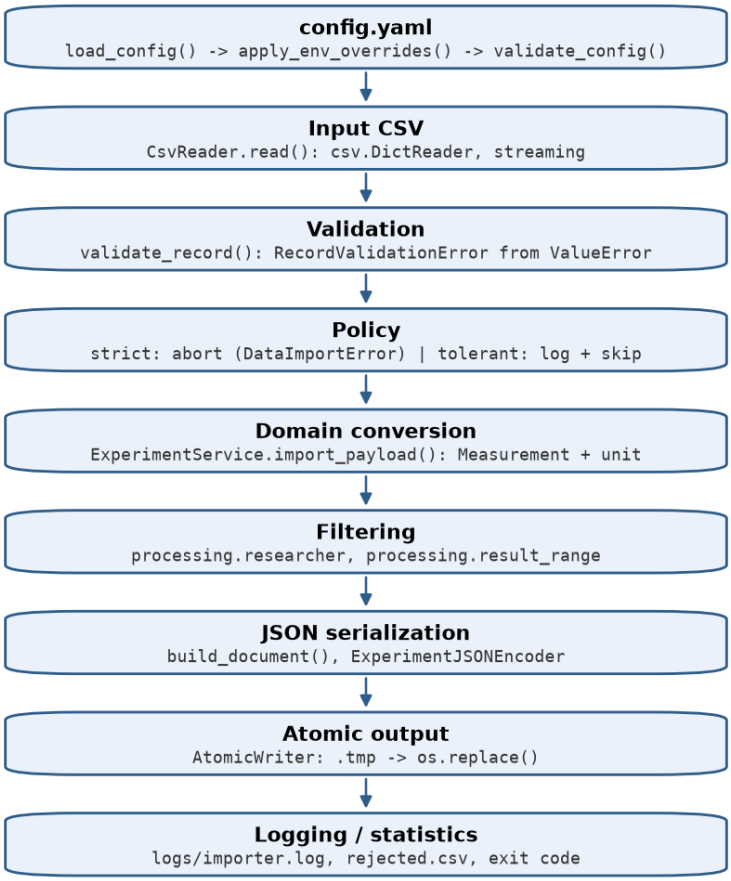


Рис. 5.1. Архітектура надійного конвеєра імпорту

4.3. Exception hierarchy і custom exceptions

Усі винятки застосунку успадковуються від `ExperimentError`, тому зовнішній код може перехопити будь-яку очікувану помилку проєкту одним `except`, не зачіпаючи програмні помилки на кшталт `TypeError`. `ConfigurationError` сигналізує про проблему конфігурації (критична, код завершення 2), `DataImportError` — про неможливість продовжити імпорт (код 1), `RecordValidationError` описує один некоректний запис і зберігає номер рядка, поле та значення, `ExportError` — проблеми запису результату.

Лістинг 5.3. Ієрархія винятків проєкту

```
ExperimentError
├── ExperimentValidationError
│   └── IncompatibleMeasurementError
├── DuplicateExperimentError
├── ExperimentNotFoundError
├── ConfigurationError
├── DataImportError
│   └── RecordValidationError (line, field, value)
└── ExportError
```

Лістинг 5.4. Нові винятки лабораторної №5

```
    """Input data cannot be read or the import must be aborted."""


class RecordValidationError(DataImportError):
    """A single input record is invalid."""
```

```
    def __init__(self, message: str, *, line: int, field: str, value: object) -> None:
        super().__init__(f"line {line}: {field}={value!r}: {message}")
        self.line = line
        self.field = field
        self.value = value


class ExportError(ExperimentError):
    """Output file cannot be written."""
```

4.4. Exception chaining

Конструкція `raise RecordValidationError(...) from error` зберігає початковий виняток у `__cause__`. У журналі й traceback видно і доменну причину («result має бути числом»), і технічну (`ValueError` від `float()`), тому діагностика не втрачає інформації.

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab05$ python benchmarks/import_experiments.py 3
=== Experiment 3: exception_chaining
Record: 1,Thermal conductivity of Al alloy,Olena Kovalenko,temperature,wrong,2026-02-03
Traceback (most recent call last):
  File "src/experiment_registry/importer/validators.py", line 39, in validate_record
    result = float(raw_result)
ValueError: could not convert string to float: 'wrong'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "benchmarks/import_experiments.py", line 122, in experiment_exception_chaining
    validate_record(row, line=2, rules=rules)
    ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "src/experiment_registry/importer/validators.py", line 41, in validate_record
    raise RecordValidationError("must be a number", line=line, field="result", value=raw_result) from
error
experiment_registry.exceptions.RecordValidationError: line 2: result='wrong': must be a number

error.__cause__ -> ValueError: could not convert string to float: 'wrong'
error.field='result', error.line=2
```

Рис. 5.2. Експеримент 3: ланцюжок ValueError → RecordValidationError

4.5. Configuration loader і config.yaml

Конфігурація описана в YAML і завантажується `yaml.safe_load` (без виконання довільних тегів). Сирий словник перетворюється на типізовані frozen dataclasses, після чого `validate_config` перевіряє значення за принципом fail-fast. Змінні середовища мають пріоритет над YAML.

Лістинг 5.5. Файл config.yaml

```
# Configuration of the experiment importer (lab 5).
version: 1

input:
  path: data/experiments.csv

output:
  path: output/experiments.json
  errors_path: output/rejected.csv   # rejected records with the reason
  create_dirs: true
  indent: 2

validation:
  # allowed parameters and their measurement units
```

```
  allowed_parameters:
    temperature: K
    pressure: kPa
    voltage: V
    frequency: Hz
    concentration: mol/L
  min_result: 0.0
  max_result: 10000.0

processing:
  skip_invalid: true        # true = tolerant mode, false = strict mode
  researcher: null          # e.g. "Olena Kovalenko"
  result_range: null        # e.g. [100, 400]

logging:
  level: INFO
  file: logs/importer.log
  max_bytes: 1000000
  backup_count: 3
```

Лістинг 5.6. Configuration loader

```
def load_config(path: Path, env: Mapping[str, str] | None = None) -> AppConfig:
    """Read YAML, apply environment overrides and return a validated configuration."""
    try:
        with path.open(encoding="utf-8") as file:
            data = yaml.safe_load(file)
    except FileNotFoundError as error:
        raise ConfigurationError(f"Configuration file not found: {path}.") from error
    except yaml.YAMLError as error:
        raise ConfigurationError(f"Invalid YAML in {path}: {error}.") from error
    if not isinstance(data, dict):
        raise ConfigurationError(f"Configuration root must be a mapping, got
{type(data).__name__}.")
    data = apply_env_overrides(data, os.environ if env is None else env)
    return parse_config(data, base_dir=path.parent)


def apply_env_overrides(data: dict[str, Any], env: Mapping[str, str]) -> dict[str, Any]:
    """Environment variables override YAML values (EXPERIMENTS_INPUT, ...)."""
    if "EXPERIMENTS_INPUT" in env:
        data.setdefault("input", {})["path"] = env["EXPERIMENTS_INPUT"]
    if "EXPERIMENTS_OUTPUT" in env:
        data.setdefault("output", {})["path"] = env["EXPERIMENTS_OUTPUT"]
    if "EXPERIMENTS_SKIP_INVALID" in env:
        data.setdefault("processing", {})["skip_invalid"] =
_env_bool(env["EXPERIMENTS_SKIP_INVALID"])
    if "EXPERIMENTS_LOG_LEVEL" in env:
        data.setdefault("logging", {})["level"] = env["EXPERIMENTS_LOG_LEVEL"]
    return data
```

Validation configuration

Лістинг 5.7. Перевірка конфігурації

```
@dataclass(frozen=True)
class ValidationConfig:
    allowed_parameters: dict[str, str]  # parameter -> unit
    min_result: float
    max_result: float


def validate_config(config: AppConfig) -> None:
    v = config.validation
    if not v.allowed_parameters:
        raise ConfigurationError("validation.allowed_parameters must not be empty.")
    if v.min_result >= v.max_result:
        raise ConfigurationError(f"min_result ({v.min_result}) must be less than max_result
({v.max_result}).")
    if config.processing.result_range is not None:
        low, high = config.processing.result_range
        if low > high:
            raise ConfigurationError(f"processing.result_range: {low} > {high}.")
    if config.logging.level not in LOG_LEVELS:
        raise ConfigurationError(f"Unknown log level {config.logging.level!r}.")
```

```
    if config.indent < 0:
        raise ConfigurationError("output.indent must be >= 0.")
    if not config.input_path.is_file():
        raise ConfigurationError(f"Input file not found: {config.input_path}.")
    if not config.create_dirs and not config.output_path.parent.is_dir():
        raise ConfigurationError(f"Output directory does not exist:
{config.output_path.parent}.")
```

4.6. Logging configuration та log file

Журнал пишеться у `logs/importer.log` через `RotatingFileHandler` (ротація за розміром, кількість резервних копій задається в YAML); на консоль виводяться лише WARNING і вище. Секрети в журнал не потрапляють: логуються лише шляхи, номери рядків і причини помилок.

Лістинг 5.8. Налаштування logging

```
def setup_logging(config: LoggingConfig, console: bool = True) -> logging.Logger:
    """Configure the importer logger; calling it again replaces old handlers."""
    logger = logging.getLogger(LOGGER_NAME)
    logger.setLevel(config.level)
    for handler in list(logger.handlers):
        logger.removeHandler(handler)
        handler.close()
    config.file.parent.mkdir(parents=True, exist_ok=True)
    file_handler = RotatingFileHandler(config.file, maxBytes=config.max_bytes,
                                       backupCount=config.backup_count, encoding="utf-8")
    file_handler.setFormatter(logging.Formatter(FORMAT))
    logger.addHandler(file_handler)
    if console:
        stream = logging.StreamHandler()
        stream.setLevel(logging.WARNING)
        stream.setFormatter(logging.Formatter("%(levelname)s: %(message)s"))
        logger.addHandler(stream)
    logger.propagate = False
    return logger
```

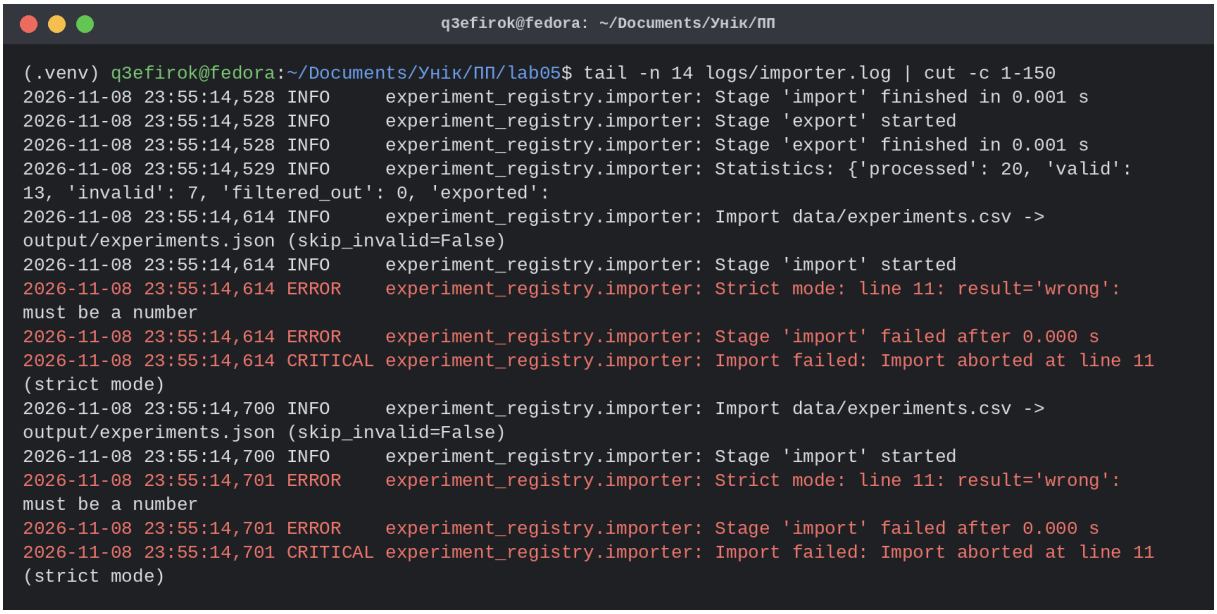


Рис. 5.3. Фрагмент logs/importer.log

4.7. Streaming reader

`CsvReader.read` — генератор: `csv.DictReader` читає файл порядково всередині `with`, тому файл закривається навіть при винятку чи достроковому завершенні. Помилки `OSError` і `UnicodeDecodeError` перетворюються на `DataImportError` зі збереженням причини. Формат обирається за розширенням (`.csv`, `.jsonl`).

Лістинг 5.9. Streaming reader

```
class CsvReader:
    """Yields (line_number, row) lazily; the file is never loaded completely."""

    def read(self, path: Path) -> Iterator[tuple[int, Row]]:
        try:
            with path.open(encoding="utf-8", newline="") as file:
                reader = csv.DictReader(file)
                missing = set(REQUIRED_COLUMNS) - set(reader.fieldnames or [])
                if missing:
                    raise DataImportError(f"{path}: missing columns {sorted(missing)}")
                for row in reader:
                    yield reader.line_num, {key: (value or "") for key, value in row.items() if
key}
        except UnicodeDecodeError as error:
            raise DataImportError(f"{path}: file is not valid UTF-8") from error
        except OSError as error:
            raise DataImportError(f"{path}: cannot read file ({error.strerror})") from error


def reader_for(path: Path) -> RecordReader:
    """Choose a reader by file extension."""
    try:
        return READERS[path.suffix.lower()]
    except KeyError:
        raise DataImportError(f"Unsupported input format: {path.suffix!r}") from None
```

4.8. Validation code

Лістинг 5.10. Перевірка запису

```
def validate_record(row: Row, line: int, rules: ValidationConfig, today: date | None = None) ->
ExperimentPayload:
    """Return a typed payload or raise RecordValidationError (with the original cause
chained)."""
    raw_id = _required(row, "experiment_id", line)
    try:
        experiment_id = int(raw_id)
    except ValueError as error:
        raise RecordValidationError("must be an integer", line=line, field="experiment_id",
value=raw_id) from error
    if experiment_id <= 0:
        raise RecordValidationError("must be positive", line=line, field="experiment_id",
value=raw_id)

    name = _required(row, "name", line)
    researcher = " ".join(_required(row, "researcher", line).split())

    parameter = _required(row, "parameter", line).lower()
    if parameter not in rules.allowed_parameters:
        raise RecordValidationError(f"allowed: {sorted(rules.allowed_parameters)}",
                                    line=line, field="parameter", value=parameter)

    raw_result = _required(row, "result", line)
    try:
        result = float(raw_result)
    except ValueError as error:
        raise RecordValidationError("must be a number", line=line, field="result",
value=raw_result) from error
    if not math.isfinite(result) or not rules.min_result <= result <= rules.max_result:
        raise RecordValidationError(f"must be within [{rules.min_result}, {rules.max_result}]",
                                    line=line, field="result", value=raw_result)
```

```
    raw_date = _required(row, "date", line)
    try:
        day = date.fromisoformat(raw_date)
    except ValueError as error:
        raise RecordValidationError("must be a date YYYY-MM-DD", line=line, field="date",
value=raw_date) from error
    if day > (today or date.today()):
        raise RecordValidationError("date is in the future", line=line, field="date",
value=raw_date)

    return ExperimentPayload(
        experiment_id=experiment_id, name=name, researcher=researcher, parameter=parameter,
        result=result, unit=rules.allowed_parameters[parameter], date=day.isoformat(),
    )
```

4.9. Strict і tolerant processing, import statistics

Політика обробки некоректних записів задається `processing.skip_invalid`. У tolerant-режимі запис журналюється як WARNING, додається до `rejected.csv` і пропускається; у strict-режимі перша помилка зупиняє імпорт винятком `DataImportError`, причиною якого є `RecordValidationError`. Статистика (`ImportStatistics`) рахує оброблені, коректні, некоректні, відфільтровані та експортовані записи і помилки за полями.

Лістинг 5.11. Статистика імпорту

```
@dataclass
class ImportStatistics:
    processed: int = 0
    valid: int = 0
    invalid: int = 0
    filtered_out: int = 0
    exported: int = 0
    errors_by_field: Counter[str] = field(default_factory=Counter)

    def as_dict(self) -> dict[str, Any]:
        data = asdict(self)
        data["errors_by_field"] = dict(self.errors_by_field)
        return data
```

Лістинг 5.12. Політика strict/tolerant і потокова перевірка

```
    def _reject(self, error: RecordValidationError) -> None:
        self.stats.invalid += 1
        self.stats.errors_by_field[error.field] += 1
        self.rejected.append({"line": str(error.line), "field": error.field,
                              "value": str(error.value), "error": str(error)})
        if not self.config.processing.skip_invalid:
            self.logger.error("Strict mode: %s", error)
            raise DataImportError(f"Import aborted at line {error.line} (strict mode)") from
error
        self.logger.warning("Skipped invalid record: %s", error)

    def iter_valid(self) -> Iterator[tuple[int, ExperimentPayload]]:
        """Lazily yield (line, payload) for valid records, applying the strict/tolerant
policy."""
        rules = self.config.validation
        for line, row in self.reader.read(self.config.input_path):
            self.stats.processed += 1
            try:
                payload = validate_record(row, line, rules)
            except RecordValidationError as error:
                self._reject(error)
                continue
            yield line, payload
```

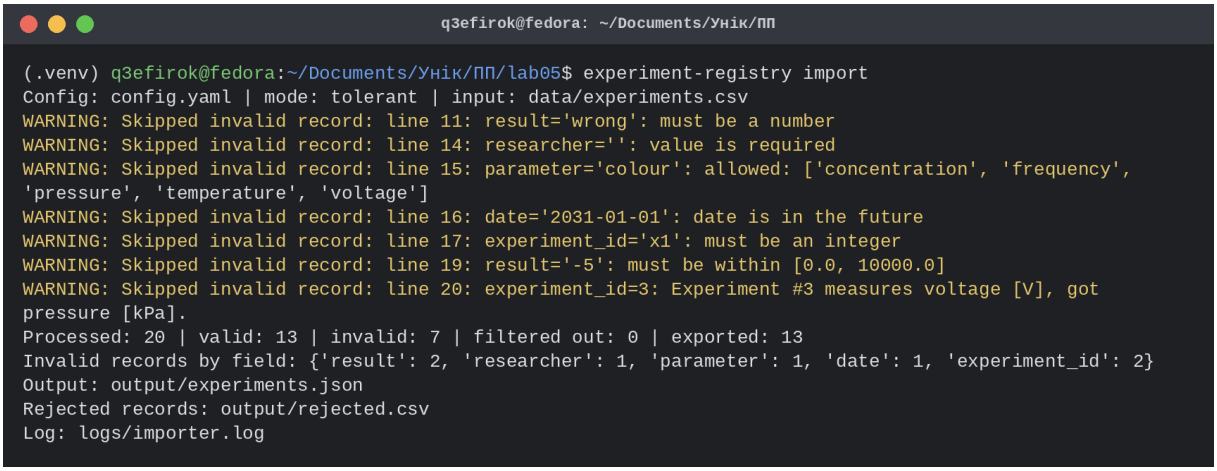


Рис. 5.4. Tolerant processing (режим за замовчуванням із config.yaml)

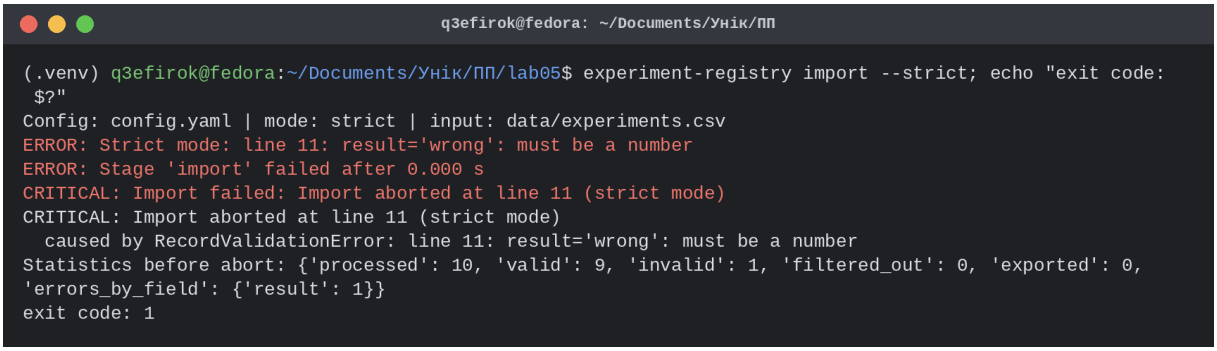


Рис. 5.5. Strict processing: імпорт зупиняється на першому некоректному записі

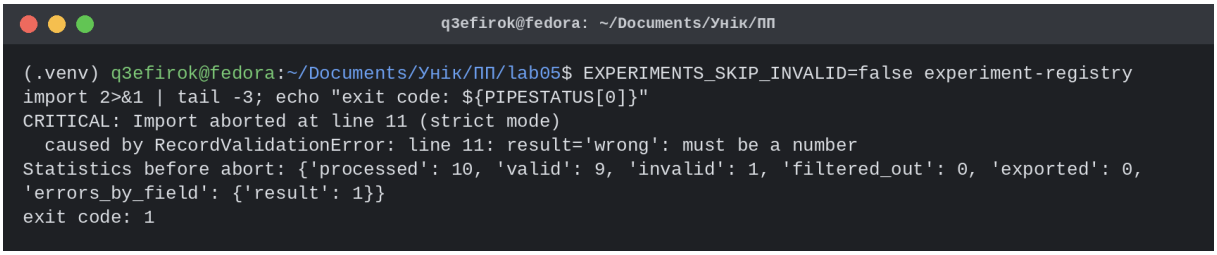


Рис. 5.6. Той самий strict-режим, увімкнений змінною середовища

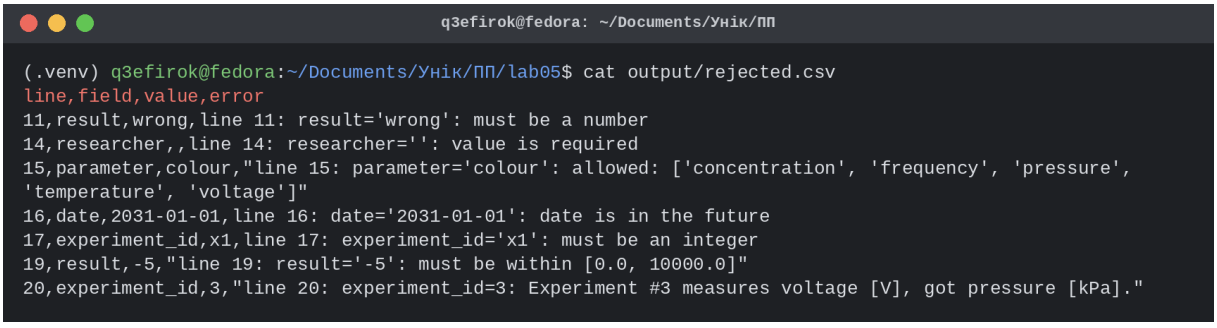


Рис. 5.7. Відхилені записи з причинами (output/rejected.csv)

4.10. Зміна поведінки через YAML без редагування коду

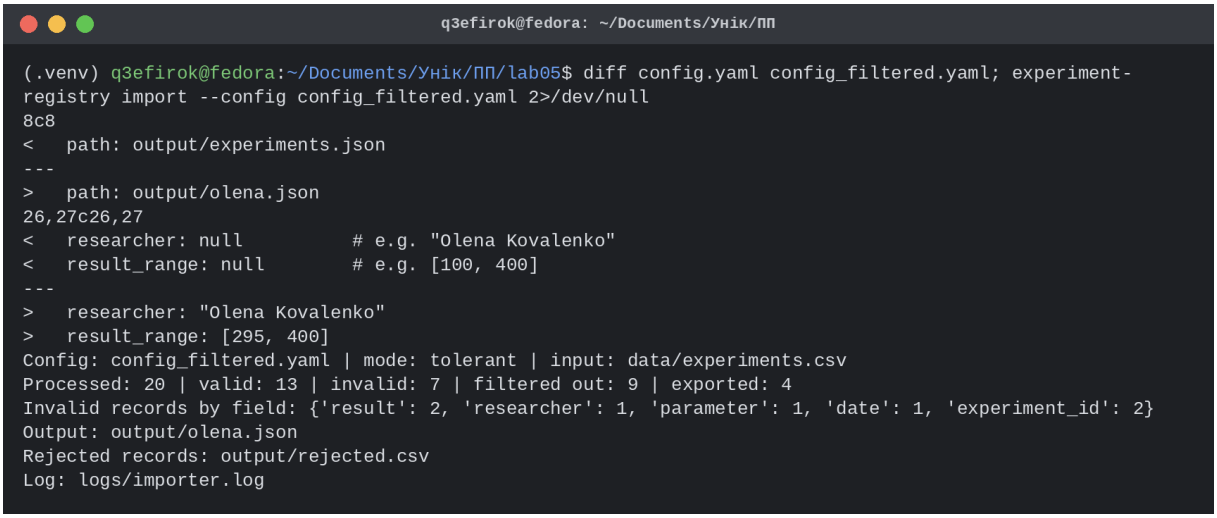


Рис. 5.8. Фільтр за дослідником і діапазоном задано лише в YAML

4.11. JSON serialization

Результат містить версію схеми, час генерації, статистику, підсумки за експериментами (стратегії average/min/max/std_dev з лабораторної №4) і список коректних вимірювань з одиницями з конфігурації. Дати серіалізуються власним `ExperimentJSONEncoder`; доменні об’єкти не серіалізуються напряму — експортуються лише payload-словники.

Лістинг 5.13. JSON serialization

```
class ExperimentJSONEncoder(json.JSONEncoder):
    """Serializes dates and other non-standard values."""

    def default(self, o: Any) -> Any:
        if isinstance(o, (date, datetime)):
            return o.isoformat()
        if isinstance(o, Path):
            return str(o)
        return super().default(o)


def export_json(path: Path, data: Any, indent: int = 2, create_dirs: bool = False) -> Path:
    """Serialize data to JSON atomically; any failure is reported as ExportError."""
    try:
        with AtomicWriter(path, create_dirs) as file:
            json.dump(data, file, indent=indent or None, ensure_ascii=False,
cls=ExperimentJSONEncoder)
            file.write("\n")
    except ExportError:
        raise
    except (TypeError, ValueError, OSError) as error:
        raise ExportError(f"JSON export to {path} failed: {error}") from error
    return path
```

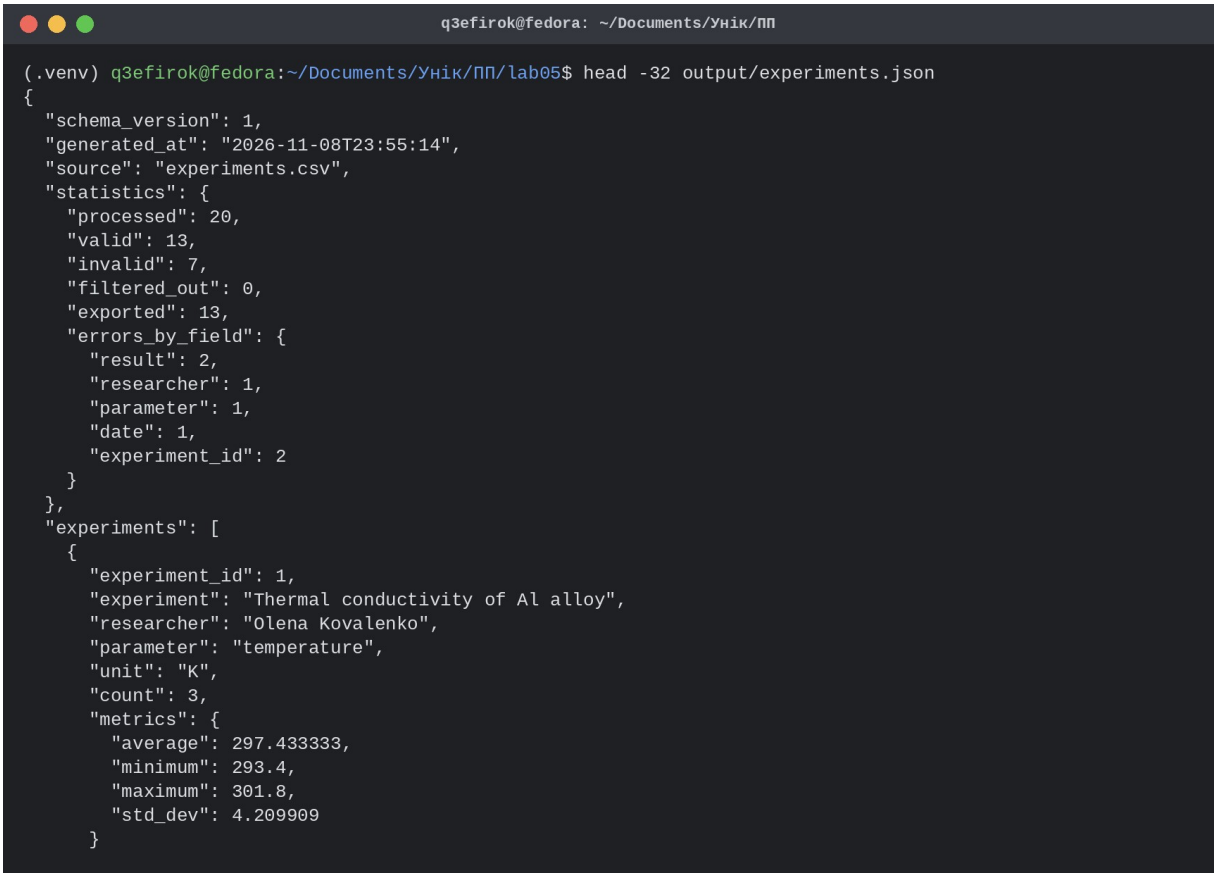


Рис. 5.9. Початок output/experiments.json



Рис. 5.10. Перший запис у розділі results

4.12. Атомарний експорт і власний context manager

`AtomicWriter` реалізує протокол context manager (`__enter__`/`__exit__`): дані пишуться у тимчасовий файл у тому самому каталозі, і лише після успішного завершення блоку `with` виконується `os.replace`, який атомарно замінює цільовий файл. Якщо в блоці виникає виняток, тимчасовий файл видаляється, `__exit__` повертає `False` і виняток поширюється далі. Другий власний context manager, `log_stage`, створено через `@contextmanager`: він журналює початок, тривалість і збій етапу.

Лістинг 5.14. Власний context manager AtomicWriter

```
class AtomicWriter:
    """Context manager: write into a temporary file, then atomically replace the target.

    If the block raises, the temporary file is removed and the previous target stays intact.
```

```
    """

    def __init__(self, target: Path, create_dirs: bool = False, encoding: str = "utf-8") ->
None:
        self.target = target
        self.create_dirs = create_dirs
        self.encoding = encoding
        self._tmp_path: Path | None = None
        self._file: IO[str] | None = None

    def __enter__(self) -> IO[str]:
        try:
            if self.create_dirs:
                self.target.parent.mkdir(parents=True, exist_ok=True)
            fd, name = tempfile.mkstemp(dir=self.target.parent, prefix=f".{self.target.name}.",
                                        suffix=".tmp")
        except OSError as error:
            raise ExportError(f"Cannot create temporary file for {self.target}") from error
        self._tmp_path = Path(name)
        self._file = os.fdopen(fd, "w", encoding=self.encoding, newline="")
        return self._file

    def __exit__(self, exc_type: type[BaseException] | None, exc: BaseException | None,
                 tb: TracebackType | None) -> Literal[False]:
        assert self._file is not None and self._tmp_path is not None
        self._file.close()
        if exc_type is not None:
            self._tmp_path.unlink(missing_ok=True)
            return False  # propagate the original exception
        try:
            os.replace(self._tmp_path, self.target)
        except OSError as error:
            self._tmp_path.unlink(missing_ok=True)
            raise ExportError(f"Cannot replace {self.target}") from error
        return False
```

Лістинг 5.15. Context manager на основі генератора

```
@contextmanager
def log_stage(logger: logging.Logger, stage: str) -> Iterator[None]:
    """Context manager that logs the start, duration and failure of a stage."""
    logger.info("Stage '%s' started", stage)
    start = time.perf_counter()
    try:
        yield
    except Exception:
        logger.error("Stage '%s' failed after %.3f s", stage, time.perf_counter() - start)
        raise
    logger.info("Stage '%s' finished in %.3f s", stage, time.perf_counter() - start)
```

4.13. Контрольоване завершення при критичних помилках

Лістинг 5.16. Обробка критичних помилок і коди завершення

```
def run_import(config_path: Path, strict: bool | None = None) -> int:
    """Run the import; return a process exit code instead of crashing with a traceback."""
    try:
        config = load_config(config_path)
    except ConfigurationError as error:
        print(f"CRITICAL: configuration error: {error}", file=sys.stderr)
        if error.__cause__ is not None:
            print(f"  caused by {type(error.__cause__).__name__}: {error.__cause__}",
file=sys.stderr)
        return EXIT_CONFIG_ERROR

    if strict is not None:
        config = replace(config, processing=replace(config.processing, skip_invalid=not strict))
    logger = setup_logging(config.logging)
    mode = "tolerant" if config.processing.skip_invalid else "strict"
    print(f"Config: {config_path} | mode: {mode} | input: {config.input_path}")

    service = ImportService(config, logger)
    try:
        stats = service.run()
```

```
    except (DataImportError, ExportError) as error:
        logger.critical("Import failed: %s", error)
        print(f"CRITICAL: {error}", file=sys.stderr)
        if error.__cause__ is not None:
            print(f"  caused by {type(error.__cause__).__name__}: {error.__cause__}",
file=sys.stderr)
        print(f"Statistics before abort: {service.stats.as_dict()}")
        return EXIT_IMPORT_FAILED

    print(f"Processed: {stats.processed} | valid: {stats.valid} | invalid: {stats.invalid} | "
          f"filtered out: {stats.filtered_out} | exported: {stats.exported}")
    if stats.errors_by_field:
        print(f"Invalid records by field: {dict(stats.errors_by_field)}")
    print(f"Output: {config.output_path}")
    if config.errors_path is not None and service.rejected:
        print(f"Rejected records: {config.errors_path}")
    print(f"Log: {config.logging.file}")
    return EXIT_OK
```

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab05$ experiment-registry import --config missing.yaml;
echo "exit code: $?"
CRITICAL: configuration error: Configuration file not found: missing.yaml.
  caused by FileNotFoundError: [Errno 2] No such file or directory: 'missing.yaml'
exit code: 2
```

Рис. 5.11. Відсутній файл конфігурації: повідомлення без traceback і код 2

5. Експериментальна частина

5.1. Експеримент 1. Strict vs tolerant

Вхідний файл містить 1000 коректних і 10 некоректних записів, рівномірно розподілених по файлу (нечисловий результат, порожній дослідник, неіснуюча дата, невідомий параметр, від'ємний ID, результат поза діапазоном, порожня назва, NaN тощо).

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab05$ python benchmarks/import_experiments.py 1
=== Experiment 1: strict_vs_tolerant
Input: data/generated/strict_tolerant.csv (1000 valid + 10 invalid records)
Mode      | processed | valid | invalid | exported | finished
strict    |        92 |    91 |       1 |        0 | aborted (exit code 1): Import aborted at line 93
(strict mode)
tolerant  |      1010 |  1000 |      10 |     1000 | normally (exit code 0)
```

Рис. 5.12. Результати експерименту strict vs tolerant

Таблиця 5.2. Порівняння strict і tolerant режимів

Параметр	Strict	Tolerant
Оброблено записів	92	1010
Valid	91	1000
Invalid	1	10
Exported	0	1000
Завершення application	перервано, код 1	нормально, код 0

Strict-режим доцільний, коли часткові дані неприпустимі: наприклад, імпорт серії калібрувальних вимірювань, де пропуск одного значення спотворить результат,

або підготовка даних для публікації. Tolerant-режим підходить для великих потоків даних з датчиків чи журналів, де кілька зіпсованих рядків не повинні блокувати обробку тисяч коректних, а відхилені записи аналізуються окремо (`rejected.csv`, журнал).

5.2. Експеримент 2. Streaming vs eager

Лістинг 5.17. Порівнювані варіанти та вимірювання

```
def eager_import(path: Path) -> list[dict[str, str]]:
    with path.open(encoding="utf-8", newline="") as file:
        return list(csv.DictReader(file))


def streaming_import(path: Path) -> Iterator[dict[str, str]]:
    with path.open(encoding="utf-8", newline="") as file:
        yield from csv.DictReader(file)


def measure(func: Callable[[], int]) -> tuple[float, float, int]:
    start = time.perf_counter()
    count = func()
    elapsed = time.perf_counter() - start
    tracemalloc.start()
    func()
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return elapsed, peak / 1024 / 1024, count
```

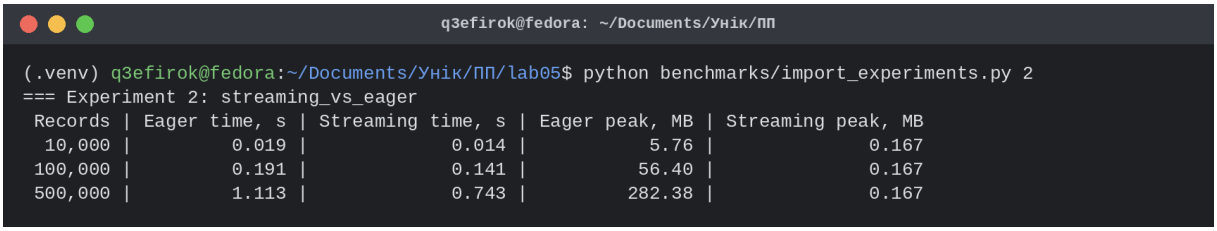


Рис. 5.13. Результати експерименту streaming vs eager

Таблиця 5.3. Час і peak memory (tracemalloc)

Records	Eager time, с	Streaming time, с	Eager peak RAM, МБ	Streaming peak RAM, МБ
10 000	0.019	0.014	5.76	0.167
100 000	0.191	0.141	56.40	0.167
500 000	1.113	0.743	282.38	0.167

Пам’ять eager-варіанта зростає лінійно з кількістю записів (до 282.38 МБ на 500 000 рядків), бо всі словники зберігаються одночасно. Streaming-варіант тримає в пам’яті один запис і використовує сталі ≈0.167 МБ. У цьому вимірюванні streaming виявився ще й трохи швидшим через менше навантаження на алокатор, але головна його перевага — передбачуване споживання пам’яті; мінімальний час виконання streaming не гарантує.

5.3. Експеримент 4. Atomic output



Рис. 5.14. Навмисна помилка під час експорту

Під час серіалізації виникає `TypeError`, частина даних уже записана у тимчасовий файл. `AtomicWriter` видаляє `.tmp`, попередній коректний `experiments.json` залишається незмінним, а помилка журналюється як `ExportError` з причиною `TypeError`.

5.4. Експеримент 5. Configuration errors

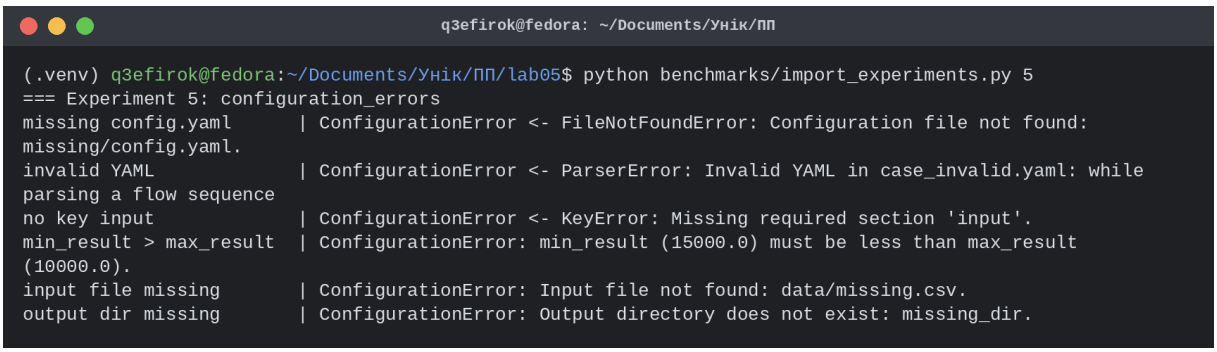


Рис. 5.15. Поведінка за некоректної конфігурації

У всіх випадках застосунок завершується контрольовано з `ConfigurationError` ще до читання даних (fail-fast). Для помилок, що походять із бібліотек (`FileNotFoundError`, `yaml.ParserError`, `KeyError`), збережено ланцюжок причин. Аналогом `minimum_grade = 150` для варіанта є `min_result = 15000`, що більше за `max_result`.

5.5. Завдання підвищеної складності

- єдиний інтерфейс читачів (`RecordReader` Protocol) і автоматичний вибір формату за розширенням (CSV, JSON Lines);
- окремий CSV з відхиленими записами (`rejected.csv`);
- configurable logging і log rotation (`RotatingFileHandler`);
- nested configuration, перевизначення через environment variables, версія схеми конфігурації;
- atomic export, власний `ExperimentJSONEncoder`, context manager `log_stage`;
- summary report з метриками за експериментами у вихідному JSON.

6. Відповіді на контрольні запитання

1. Що таке exception?

Exception — це об’єкт, що сигналізує про помилкову або виняткову ситуацію під час виконання програми. Коли він виникає, нормальний потік виконання переривається, і Python шукає найближчий відповідний обробник except вгору по стеку викликів; якщо обробника немає, програма завершується з traceback. Стандартні винятки є екземплярами класів, похідних від BaseException (зазвичай від Exception).

2. Для чого використовується try?

Блок try позначає ділянку коду, у якій може виникнути exception і яку потрібно контролювати: якщо всередині станеться виняток, виконання переходить до відповідного except. У try варто розміщувати лише потенційно небезпечну операцію, наприклад float(row["result"]) під час імпорту CSV, а не великий фрагмент програми, щоб не перехопити зайвого.

3. Для чого except?

Блок except перехоплює та обробляє exception певного типу або кортежу типів, що виник у try, наприклад except ValueError as exc. Обробник може залогувати помилку, підставити значення за замовчуванням, підняти власний виняток (raise RecordValidationError(...) from exc) або пропустити запис у tolerant mode. Обробники перевіряються зверху вниз, тому специфічніші типи пишуть першими.

4. Коли виконується else після try?

Блок else виконується лише тоді, коли код у try завершився без жодного exception, і до блоку finally. У нього виносять дії, які мають відбутися тільки після успішного try, але помилки яких не повинен перехоплювати той самий except, наприклад додавання успішно розібраного Measurement до результатів.

5. Коли виконується finally?

Блок finally виконується завжди: після успішного try, після обробленого або необробленого exception і навіть при return, break чи continue всередині try. Його використовують для гарантованого звільнення ресурсів — закриття файлів і з’єднань, видалення тимчасових файлів, хоча для більшості таких випадків зручніший with.

6. Для чого використовується raise?

Оператор raise явно піднімає exception, наприклад raise ConfigurationError("Unknown parameter") при некоректній конфігурації. Так функція сигналізує, що не може виконати свій контракт, замість повернення None чи «магічних» значень; також raise використовують, щоб перетворити низькорівневу помилку на доменну.

7. Що означає raise без аргументів?

Голий raise усередині блоку except повторно піднімає щойно перехоплений exception зі збереженням його оригінального traceback. Це використовують, коли треба виконати певну дію (залогувати, прибрати ресурс), але не «проковтнути»

виняток, а передати його вище по стеку. Поза обробником голий raise спричиняє RuntimeError, бо немає активного винятку.

8. Чому не рекомендується except: pass?

Конструкція except: pass мовчки «ковтає» будь-які винятки, включно з KeyboardInterrupt, SystemExit і помилками програміста на кшталт NameError чи TypeError. У результаті дефекти маскуються, програма продовжує роботу в некоректному стані без жодного сліду в logs, і знайти причину збою потім дуже складно. Натомість слід перехоплювати конкретний тип і хоча б логувати подію.

9. Чому потрібно перехоплювати конкретні exception types?

Перехоплення конкретних типів (ValueError, KeyError, FileNotFoundError) гарантує, що обробляються лише очікувані ситуації, для яких є осмислена реакція, а неочікувані помилки, тобто баги, не маскуються і проявляються явно. Крім того, різні помилки потребують різної обробки: відсутній файл конфігурації — ConfigurationError, нечислове значення result — RecordValidationError.

10. Що таке exception hierarchy?

Exception hierarchy — це дерево класів винятків, пов’язаних наслідуванням: у Python корінь BaseException, далі Exception, від якого походять ValueError, OSError, LookupError (KeyError, IndexError) тощо; перехоплення базового класу ловить і всі підкласи. У проєкті є власна ієрархія: ExperimentError → ConfigurationError, DataImportError → RecordValidationError, ExportError, тож except ExperimentError перехоплює всі доменні помилки пакета.

11. Для чого створювати custom exceptions?

Custom exceptions дають предметно-орієнтовані зрозумілі назви помилок і дозволяють викликачу перехоплювати саме помилки пакета (except DataImportError), не зачіпаючи інші. Вони можуть містити додатковий контекст, наприклад номер рядка CSV чи назву поля, і приховують деталі реалізації: замість ValueError від float() користувач отримує RecordValidationError.

12. Що таке exception chaining?

Exception chaining — це зв’язування нового exception з початковим, що його спричинив. Python зберігає оригінал в атрибуті __cause__ при явному chaining (raise ... from ...) або в __context__ при неявному, коли виняток виник під час обробки іншого. У traceback показуються обидва винятки з поясненням «The above exception was the direct cause of the following exception» або «During handling of the above exception, another exception occurred».

13. Що означає raise ... from ...?

Запис raise NewError(...) from exc піднімає новий виняток і явно вказує exc як його безпосередню причину, записуючи його в __cause__. Наприклад, в імпортері ValueError від float(row["result"]) перетворюється на RecordValidationError з номером рядка: raise RecordValidationError(...) from exc. Форма raise ... from None, навпаки, приховує початковий контекст.

14. Яку перевагу дає chaining для debugging?

Chaining зберігає повний ланцюг причин: розробник бачить і високорівневу доменну помилку з контекстом (файл, рядок, поле), і первинний низькорівневий exception з оригінальним traceback. Інформація про справжню причину збою не втрачається, а верхні рівні водночас працюють зі зрозумілими власними типами; у тестах причину можна перевірити через exc_info.value.__cause__.

15. Що означає EAFP?

EAFP (Easier to Ask Forgiveness than Permission) — типовий для Python підхід: спочатку виконати операцію, а помилку обробити через try/except, якщо вона виникне. Наприклад, try: value = float(row["result"]) except ValueError: ... замість попередньої перевірки рядка. Такий код лаконічний і уникає race condition між перевіркою та дією.

16. Що означає LBYL?

LBYL (Look Before You Leap) — підхід, за якого перед операцією явно перевіряють передумови, наприклад if "result" in row: або if path.exists():. Він читабельний для очікуваних ситуацій, але може дублювати логіку та має race condition: стан може змінитися між перевіркою і дією, наприклад файл видалять після exists(). Вибір між EAFP і LBYL залежить від контексту.

17. Що таке fail-fast?

Fail-fast — принцип, за яким програма виявляє некоректний стан якомога раніше й одразу завершує операцію з чітким exception, а не продовжує роботу з поганими даними. Наприклад, якщо YAML-конфігурація не містить дозволених параметрів або вхідний файл не існує, одразу піднімається ConfigurationError ще до початку тривалого імпорту.

18. Де доцільно застосовувати fail-fast?

Fail-fast доцільний під час старту застосунку (перевірка configuration, наявності файлів, обов’язкових параметрів), у конструкторах і валідації domain objects (наприклад, Measurement не можна створити без unit), на межах системи при прийомі вхідних даних і в strict mode імпорту. Менш доречний він там, де потрібно обробити максимум даних і зібрати звіт про помилки, як у tolerant mode.

19. Що таке context manager?

Context manager — це об’єкт, що керує життєвим циклом ресурсу в межах блоку with: виконує дії на вході (відкрити файл, захопити lock, почати транзакцію) і гарантовано виконує завершальні дії на виході (закрити, звільнити, відкотити), навіть якщо виник exception. Він реалізує протокол __enter__/__exit__ або створюється через contextlib.contextmanager.

20. Для чого використовується with?

Оператор with використовується для безпечної роботи з ресурсами через context manager: він викликає __enter__ на початку блоку і __exit__ при виході з нього за будь-яких умов. Це замінює громіздкий try/finally; наприклад, with path.open("r", encoding="utf-8", newline="") as file: при читанні CSV гарантує закриття файлу.

21. Які methods утворюють context manager protocol?

Context manager protocol утворюють два спеціальні методи: __enter__(self), який викликається при вході в блок with, і __exit__(self, exc_type, exc_value, traceback), який викликається при виході з нього. Для асинхронних context managers (async with) аналогічну роль виконують __aenter__ та __aexit__.

22. Що робить __enter__()?

Метод __enter__() виконує підготовку ресурсу на початку блоку with — відкриває файл, створює тимчасовий файл, починає транзакцію — і повертає значення, яке присвоюється змінній після as. Найчастіше він повертає сам ресурс або self.

23. Що робить __exit__()?

Метод __exit__(exc_type, exc_value, traceback) викликається при виході з блоку with і звільняє ресурс; якщо блок завершився винятком, у параметри передається інформація про нього, інакше всі три дорівнюють None. Якщо метод повертає True, exception пригнічується, а при False чи None поширюється далі. У власному context manager, наприклад, саме тут можна видалити тимчасовий файл у разі помилки.

24. Чи закриває with файл при exception?

Так. Файловий об’єкт є context manager, і його __exit__ викликається за будь-якого виходу з блоку with — нормального, через exception, return чи break, — тому файл гарантовано закривається. Сам exception при цьому не пригнічується і після закриття файлу поширюється далі.

25. Для чого contextlib.contextmanager?

Декоратор contextlib.contextmanager дозволяє створити context manager із generator function замість написання класу з __enter__ і __exit__. Код до yield виконується на вході, значення yield передається в змінну as, а код після yield, зазвичай у finally, — на виході, що робить реалізацію коротшою й читабельнішою.

26. Яку роль має yield у generator-based context manager?

yield розділяє generator-based context manager на дві частини: код до yield — setup (аналог __enter__), значення yield потрапляє у змінну після as, а код після yield — teardown (аналог __exit__). Якщо в блоці with виникає exception, він повторно піднімається в точці yield, тому для гарантованого cleanup yield обгортають у try/finally; generator має виконати yield рівно один раз.

27. Що таке pathlib.Path?

pathlib.Path — об’єктно-орієнтоване представлення шляху у файловій системі зі стандартної бібліотеки. Він підтримує оператор / для побудови шляхів, властивості name, stem, suffix, parent і методи exists(), open(), read_text(), write_text(), mkdir(), with_suffix(), replace(), unlink(), тому шляхи до CSV, YAML та JSON у проєкті зручно передавати як Path.

28. Чому pathlib кращий за ручну конкатенацію шляхів?

pathlib автоматично використовує правильний для ОС роздільник шляху і не допускає типових помилок рядкової конкатенації, як-от подвійних чи пропущених слешів. Крім того, Path поєднує в одному об’єкті розбір шляху і файлові операції,

робить код читабельнішим (base_dir / "data" / "experiments.csv") і краще типізується, ніж звичайні рядки.

29. Для чого задається encoding="utf-8"?

Явне encoding="utf-8" гарантує, що текст читається й записується в однаковому кодуванні незалежно від ОС і локалі, адже кодування за замовчуванням платформозалежне (наприклад, cp1251 у Windows). Без нього кириличні імена дослідників чи назви експериментів можуть спотворитися або спричинити UnicodeDecodeError.

30. Що таке CSV?

CSV (Comma-Separated Values) — простий текстовий табличний формат, у якому кожен рядок є записом, а поля розділені комою або іншим delimiter, часто з першим рядком-заголовком. Усі значення в CSV є рядками, тому їх потрібно перетворювати та валідувати; у проєкті вхідний CSV має колонки experiment_id,name,researcher,parameter,result,date.

31. Для чого csv.reader?

csv.reader читає CSV-файл і повертає iterator, кожен елемент якого — список рядків (list[str]) для одного запису, з коректною обробкою лапок, роздільників і екранування. Доступ до полів відбувається за індексом, тому він підходить для простих файлів без заголовка або з фіксованим порядком колонок.

32. Для чого csv.DictReader?

csv.DictReader читає CSV і повертає кожен рядок як dict, ключі якого — назви колонок із першого рядка (header) або з параметра fieldnames. Він працює як lazy iterator, тому дозволяє потоково обробляти записи, звертаючись до полів за назвою, наприклад row["parameter"] чи row["result"].

33. Чим DictReader зручний для structured data?

DictReader дає доступ до полів за назвою, а не за індексом, тому код читабельніший і не ламається при зміні порядку колонок. Також легко перевірити наявність обов’язкових колонок через reader.fieldnames, сформувати повідомлення про помилку з назвою поля, а сам dict природно перетворюється на domain object.

34. Для чого csv.DictWriter?

csv.DictWriter записує у CSV словники, зіставляючи їхні ключі із заданим списком fieldnames, який визначає порядок колонок. Метод writeheader() записує рядок заголовка, а writerow() і writerows() — дані; файл слід відкривати з newline="" та encoding="utf-8".

35. Що таке JSON?

JSON (JavaScript Object Notation) — текстовий мовно-незалежний формат обміну даними, що підтримує objects, arrays, strings, numbers, true/false і null; у Python їм відповідають dict, list, str, int/float, bool і None. Він є стандартом для web API та експорту даних; у проєкті валідні результати експериментів експортуються саме в JSON.

36. Що таке serialization?

Serialization — перетворення об’єкта з пам’яті у формат, придатний для збереження або передавання (рядок, байти, файл), наприклад JSON, YAML чи CSV. Наприклад, Experiment із його результатами спершу перетворюють на dict із простих типів, а потім записують через json.dump; дати зазвичай потрібно явно конвертувати в рядки.

37. Що таке deserialization?

Deserialization — зворотний процес: відновлення структур даних або об’єктів зі збереженого формату (JSON, YAML, CSV). Після десеріалізації дані обов’язково валідують, бо зовнішнє джерело може містити некоректні значення; наприклад, рядок CSV спочатку стає dict, а після перевірки — Measurement.

38. Чим json.dump() відрізняється від json.dumps()?

json.dump(obj, file) серіалізує об’єкт і одразу записує результат у відкритий файловий об’єкт, а json.dumps(obj) повертає JSON як рядок str. Обидві функції приймають однакові параметри, наприклад indent=2 та ensure_ascii=False для читабельного запису кирилиці; dumps зручний, коли рядок потрібно далі записати атомарно або передати мережею.

39. Чим json.load() відрізняється від json.loads()?

json.load(file) читає та десеріалізує JSON із відкритого файлового об’єкта, а json.loads(s) — з рядка (str, bytes або bytearray). Наприклад, load підходить для читання цілого JSON-файлу, а loads — для розбору окремих рядків JSON Lines.

40. Що таке JSONDecodeError?

json.JSONDecodeError — exception, який піднімають json.load() і json.loads(), коли вхідний текст не є коректним JSON; це підклас ValueError. Він містить атрибути msg, lineno, colno та pos, тому його зручно перехоплювати й перетворювати на власний виняток з контекстом, наприклад raise DataImportError(...) from exc.

41. Для чого використовується YAML?

YAML — людиночитаний формат серіалізації даних, що використовує відступи замість дужок і підтримує коментарі, словники, списки та скалярні типи. Найчастіше він застосовується для configuration files; у проєкті YAML задає дозволені параметри з одиницями вимірювання, межі result і режим skip_invalid. У Python його читають бібліотекою PyYAML.

42. Чому рекомендується yaml.safe_load()?

yaml.safe_load() створює лише стандартні типи Python (dict, list, str, int, float, bool, None, дати), тоді як yaml.load() з небезпечним Loader через спеціальні теги на кшталт !!python/object може створювати довільні об’єкти й виконувати код. Тому safe_load захищає від виконання шкідливого коду при читанні недовіреного файлу конфігурації.

43. Які типові сценарії використання JSON?

JSON типово використовують для обміну даними між сервісами (REST API, web-застосунки), експорту та імпорту структурованих даних, збереження результатів

обробки, кешів і повідомлень у чергах, а також для машинно згенерованих налаштувань. У проєкті в JSON атомарно експортуються валідні результати експериментів, бо цей формат легко читається будь-якою мовою.

44. Які типові сценарії використання YAML?

YAML типово застосовують для configuration files, які редагують люди: налаштування застосунків, CI/CD pipelines (наприклад, GitHub Actions), Docker Compose, Kubernetes manifests, Ansible playbooks. У проєкті YAML описує дозволені параметри та їхні одиниці, межі result і режим strict/tolerant.

45. Що таке configuration file?

Configuration file — зовнішній файл (YAML, TOML, JSON, INI), що містить налаштування поведінки програми без зміни її коду: шляхи, пороги, дозволені значення, режими роботи. Наприклад, YAML-конфігурація проєкту визначає перелік дозволених параметрів з одиницями, межі результатів і skip_invalid.

46. Чому configuration краще не hard-code у Python?

Винесена configuration дозволяє змінювати поведінку програми (межі результатів, дозволені параметри, шляхи) без редагування і повторного розгортання коду, мати різні налаштування для різних середовищ і тестів та відокремлює політику від логіки. Крім того, секрети не потрапляють у репозиторій, а код легко тестувати з різними конфігураціями.

47. Що потрібно перевіряти у configuration?

Потрібно перевіряти наявність обов’язкових ключів, типи значень, допустимі діапазони, логічну узгодженість (наприклад, мінімум менший за максимум), допустимість enum-подібних значень, непорожність списків і існування шляхів до файлів. У проєкті, наприклад, перевіряється, що кожен дозволений параметр має unit, межі result є числами, а skip_invalid — bool; при порушенні одразу піднімається ConfigurationError.

48. Що таке logging?

Logging — механізм запису подій роботи програми з рівнем важливості, часовою міткою, назвою модуля та іншим контекстом у різні призначення: консоль, файл, системний журнал. У Python це стандартний модуль logging з loggers, handlers, formatters і levels; у проєкті некоректні записи CSV та хід імпорту логуються у файл.

49. Чим logging відрізняється від print()?

print() просто виводить текст у stdout, а logging підтримує рівні важливості з фільтрацією, форматування з часом і назвою модуля, кілька handlers (файл, консоль), централізоване налаштування та вимкнення без зміни коду. Logs призначені для діагностики й моніторингу, а print — для виводу користувачеві; бібліотечний код не повинен засмічувати stdout.

50. Які існують logging levels?

Стандартні рівні за зростанням важливості: DEBUG (10) — детальна діагностика, INFO (20) — штатні події, WARNING (30) — неочікувана ситуація, але робота триває, ERROR (40) — операцію не вдалося виконати, CRITICAL (50) — критичний

збій застосунку. Також існує NOTSET (0); logger обробляє повідомлення не нижче встановленого рівня, для root за замовчуванням це WARNING.

51. Для чого logger.info()?

logger.info() записує інформаційне повідомлення про штатний перебіг роботи: початок і завершення імпорту, кількість оброблених записів, шлях до створеного файлу. Наприклад, після імпорту в проєкті доречно залогувати статистику valid/invalid записів.

52. Для чого logger.warning()?

logger.warning() фіксує неочікувану або потенційно проблемну ситуацію, яка не зупиняє роботу програми. У tolerant mode саме warning доречний при пропуску некоректного рядка CSV із зазначенням номера рядка й причини, після чого імпорт продовжується.

53. Для чого logger.error()?

logger.error() записує повідомлення про помилку, через яку певну операцію не вдалося виконати, наприклад неможливо записати експортний JSON-файл. На відміну від logger.exception(), він за замовчуванням не додає traceback, якщо не передати exc_info=True.

54. Для чого logger.exception()?

logger.exception() логує повідомлення з рівнем ERROR і автоматично додає traceback поточного exception, тому його викликають лише всередині блоку except. Це зручно для діагностики неочікуваних помилок, наприклад збою атомарного експорту чи непередбаченої помилки імпорту.

55. Що містить traceback?

Traceback містить тип і повідомлення exception, а також стек викликів від точки входу до місця виникнення помилки: для кожного frame — ім’я файлу, номер рядка, назву функції та зазвичай сам рядок коду. При exception chaining у traceback показуються також попередні винятки з __cause__ або __context__.

56. Чому secrets не можна записувати в logs?

Logs часто зберігаються довго, пересилаються в системи моніторингу, доступні багатьом людям і захищені слабше, ніж сховища секретів, тому паролі, tokens, API keys чи персональні дані з них легко витікають. Це порушує безпеку та вимоги захисту даних; замість секретів слід логувати ідентифікатори або маскувати значення.

57. Що таке streaming I/O?

Streaming I/O — обробка даних послідовно невеликими порціями (рядок за рядком, запис за записом) під час читання чи запису, без завантаження всього файлу в пам’ять. У Python її забезпечують iterators і generators, наприклад for row in csv.DictReader(file) або yield from reader у функції імпорту.

58. Яка перевага streaming для великих files?

При streaming споживання пам’яті залишається приблизно сталим і не залежить від розміру файлу, тому можна обробляти файли, більші за доступну RAM. Обробка

перших записів починається одразу, а в strict mode помилка виявляється без читання всього файлу; водночас загальний час виконання streaming не обов’язково менший.

59. Що таке eager file loading?

Eager file loading — підхід, за якого весь файл одразу повністю зчитується в пам’ять (file.read(), readlines(), list(reader), json.load), і лише потім дані обробляються. Він простий і дає довільний доступ до даних, але споживання пам’яті зростає пропорційно розміру файлу.

60. Чому list(reader) може бути небажаним?

list(reader) матеріалізує всі рядки CSV у списку в пам’яті, тому для великого файлу це означає великий memory usage аж до MemoryError і затримку перед початком обробки. До того ж втрачається перевага lazy iteration: навіть якщо strict mode зупиниться на першому некоректному рядку, весь файл уже буде прочитано.

61. Що таке strict validation mode?

Strict validation mode — режим, у якому перший некоректний запис одразу перериває весь імпорт піднятим exception (у проєкті RecordValidationError), тож некоректні дані не можуть непомітно потрапити в результат. У проєкті цей режим відповідає skip_invalid: false у YAML-конфігурації.

62. Що таке tolerant mode?

Tolerant mode — режим, у якому некоректний запис не зупиняє імпорт: він логується як warning, лічильник invalid збільшується, запис пропускається, і обробка триває. Наприкінці формується статистика valid/invalid; у проєкті режим вмикається параметром skip_invalid: true у YAML.

63. Коли доцільно використовувати strict mode?

Strict mode доцільний, коли цілісність даних критична і часткове завантаження неприпустиме: фінансові операції, медичні чи наукові дані, де пропущене вимірювання спотворить аналіз, міграції та імпорт довідників. Він також корисний під час розробки й тестування, щоб одразу виявляти проблеми з форматом даних.

64. Коли tolerant mode?

Tolerant mode доцільний для великих або «брудних» наборів даних, де окремі некоректні рядки очікувані, а важливо обробити якомога більше валідних записів: масовий імпорт із зовнішніх джерел, логи, первинне завантаження результатів вимірювань. Обов’язкова умова — логування та підрахунок пропущених записів, щоб помилки не губилися.

65. Що таке atomic write?

Atomic write — спосіб запису, за якого цільовий файл або залишається в попередньому стані, або повністю замінюється новим вмістом, без проміжного частково записаного стану. Зазвичай дані спочатку пишуть у тимчасовий файл у тій самій директорії, а потім атомарно замінюють ним цільовий через os.replace(), як в експорті JSON у проєкті.

66. Для чого використовується temporary file?

Temporary file — проміжне місце для запису: увесь вміст спочатку повністю записується в нього, і лише після успіху він замінює цільовий файл. Так споживачі ніколи не побачать напівзаписаний output, а при збої оригінальний файл залишиться неушкодженим; створювати його слід у тій самій директорії, щоб os.replace() був атомарним.

67. Що може статися при звичайному output write та аварійному завершенні?

При звичайному відкритті в режимі "w" файл одразу обрізається до нуля, тому аварійне завершення посеред запису (exception, kill, збій живлення) залишить порожній або частково записаний файл, наприклад невалідний JSON. Попередні коректні дані вже втрачено, а наступні споживачі отримають JSONDecodeError або неповні результати.

68. Навіщо видаляти temporary file після error?

Після помилки тимчасовий файл містить неповні або некоректні дані, тому його видаляють (наприклад, temporary.unlink(missing_ok=True)), щоб не залишати сміття на диску, не сплутати його з валідним результатом і не заважати наступним запускам. Після очищення помилку піднімають далі, у проєкті — як ExportError з chaining.

69. Що таке JSON Lines?

JSON Lines (NDJSON, розширення .jsonl) — текстовий формат, у якому кожен рядок файлу є окремим повноцінним JSON-значенням, зазвичай object, а записи розділені символом нового рядка. На відміну від JSON-масиву, у файлу немає спільних зовнішніх дужок, тому його можна читати й дописувати по одному запису.

70. Чому JSON Lines зручний для streaming?

Кожен рядок JSON Lines розбирається незалежно через json.loads(line), тому файл можна обробляти по одному запису без завантаження всього масиву в пам’ять, а нові записи легко дописувати в кінець. Помилка в одному рядку не робить некоректним увесь файл: її можна залогувати з номером рядка і продовжити обробку.

7. Висновок

У лабораторній роботі до проєкту experiment-registry додано надійний модуль імпорту та експорту результатів експериментів. Конфігурація винесена в YAML і перевіряється за принципом fail-fast, допустимі параметри та їх одиниці задаються без зміни коду, а змінні середовища дозволяють перевизначати режим роботи. CSV читається потоково, кожен запис проходить валідацію і перетворення в доменні об’єкти, результати серіалізуються в JSON атомарно, а журнал і `rejected.csv` фіксують усі відхилені записи.

Власна ієрархія винятків з exception chaining зберігає повну причину помилки, а коди завершення роблять поведінку передбачуваною для автоматизації. Експерименти показали: strict-режим зупинився на першому з 10 некоректних

записів, tolerant експортував 1000 коректних записів з 1010; streaming-імпорт використовує сталі ≈0.167 МБ проти 282.38 МБ для eager на 500 000 записів; атомарний запис зберігає попередній результат при збої, а помилки конфігурації не призводять до аварійного завершення з traceback.